



UNIVERSIDAD
CATÓLICA
BOLIVIANA

INFORME PRIMERA EVALUACIÓN

ASIGNATURA INTERNET DE LAS COSAS

GRUPO N.º

INTEGRANTES

1. _____
2. _____
3. _____

DOCENTE

Ing. Pamela Valenzuela

CARRERA

Ingeniería de Sistemas

FECHA DE ENTREGA

GESTIÓN

II/2026

**LA PAZ – BOLIVIA
2026**

ÍNDICE

1. Introducción	1
1.1 Descripción general del problema	1
1.2 Las tres series matemáticas seleccionadas	1
1.3 Utilidad de cada serie y su aplicación en el sistema	2
1.4 Interacción entre cliente, servidor, base de datos y dashboard	2
2. Objetivos	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
3. Diseño de la solución cliente-servidor	3
3.1 Arquitectura propuesta	3
3.2 Flujo de datos	5
3.3 Identificación del usuario correspondiente a cada cliente	5
3.4 El número de registro n	5
4. Diseño de la base de datos	6
4.1 Nombre y tecnología	6
4.2 Estructura de tablas	6
4.3 Campos y tipos de datos de la tabla central	7
4.4 Restricciones y reglas aplicadas	8
4.5 Diagrama entidad-relación	8
5. Desarrollo de la aplicación	10
5.1 Tecnologías, librerías y herramientas	10
5.2 Implementación del cliente	10
5.3 Implementación del servidor	11
5.4 Implementación de las tres series matemáticas	12
5.5 Magnitudes derivadas en el backend	14
5.6 Inserción de datos: conexión, operación y manejo de errores	15
5.7 Lectura y recuperación de datos	16
5.8 Otras funciones implementadas	16
6. Desarrollo del dashboard	17
6.1 Diseño y herramientas	17
6.2 Páginas y visualizaciones	17
6.3 Visualización del error por señal	23
6.4 Actualización en tiempo real	23
6.5 Interactividad	24
7. Pruebas y resultados	24
7.1 Comunicación cliente-servidor	24
7.2 Generación y almacenamiento de las tres series	25
7.3 Registros por usuario y por valor de n	26
7.4 Capturas del dashboard con los datos y el error	26
7.5 Análisis de los resultados	27

8. Conclusiones	27
9. Anexos y entregables	28
Lista de verificación antes de la entrega	29
Verificación técnica del sistema	29

DASHBOARD IoT DE MONITOREO PARA ROBOT ASPIRADOR

Resumen. Se desarrolló una aplicación cliente-servidor completa en Next.js 15 (App Router) con TypeScript, que cumple la función de dashboard de monitoreo en la nube para un robot aspirador tipo Roomba basado en ESP32. El cliente publica telemetría contra un endpoint HTTP con el mismo contrato de carga útil que usará MQTT; el servidor valida, calcula magnitudes derivadas, evalúa umbrales, dispara eventos y persiste en una base de datos SQLite mediante Prisma. Los datos de sensor se generan con tres aproximaciones por **series de Taylor** (seno, coseno y arcotangente), de modo que cada lectura almacena su valor aproximado, su valor real y su error, y el dashboard presenta ese error como la incertidumbre de la medición.

1. Introducción

1.1 Descripción general del problema

Un robot aspirador doméstico opera sin supervisión y sin pantalla propia: el usuario no sabe cuánto limpió, si se atascó, qué tan sucia estaba la habitación ni si la batería alcanzará para terminar. El problema abordado es la **telemetría remota**: capturar en tiempo real las lecturas de los sensores embarcados, enviarlas a un servidor, almacenarlas de forma consultable y presentarlas en un tablero que permita tanto el monitoreo en vivo como el análisis histórico. La solución propuesta es una aplicación web full-stack que implementa las cuatro capas del flujo IoT clásico —adquisición, transporte, persistencia y visualización— con las siguientes decisiones de alcance:

- **Un solo dispositivo.** No hay multi-tenant: la interfaz habla de «el robot» en singular. El identificador de cliente existe igualmente en el modelo de datos (`deviceId`), de modo que agregar dispositivos no requiere rediseñar el esquema.
- **Sin autenticación.** El dashboard se ejecuta en un entorno local controlado.
- **Sin mapa ni localización espacial.** El robot no dispone de LiDAR, cámara ni SLAM; por lo tanto no se genera planta del cuarto, trayectoria 2D ni mapa de calor espacial. Toda métrica presentada se deriva únicamente de sensores realmente presentes.
- **Prototipo físico en construcción.** El backend funciona hoy con un generador de datos simulados que publica contra el mismo endpoint que usará el ESP32 real. El frontend no distingue el origen de los datos, y apagar el simulador con una variable de entorno deja la aplicación funcional a la espera de datos reales.

1.2 Las tres series matemáticas seleccionadas

Se eligieron tres desarrollos en serie de Taylor/Maclaurin, todos de términos alternados, porque permiten observar con claridad la convergencia y acotar el error:

a) Seno

$$\sin(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

b) Coseno

$$\cos(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k}}{(2k)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

c) Arcotangente (serie de Gregory–Leibniz)

$$\arctan(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{2k+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots$$

1.3 Utilidad de cada serie y su aplicación en el sistema

El criterio de asignación no fue arbitrario: cada serie se eligió porque **su forma funcional imita el comportamiento físico de la señal** que representa.

Serie	Aplicación y justificación
Seno	Modela las tres distancias ultrasónicas (HC-SR04 frontal, izquierda y derecha). Un robot que recorre una habitación ve la distancia a los obstáculos crecer y decrecer de forma periódica al aproximarse y alejarse de paredes y muebles. Cada sensor usa un desfase propio (0, 1.1 y 2.3 rad) para que las tres señales estén correlacionadas pero nunca sincronizadas, tal como ocurriría físicamente. Además la serie del seno converge para todo x real, lo que la hace segura en todo el dominio de la simulación.
Coseno	Modela la densidad de polvo del sensor Sharp GP2Y1010AU0F. La oscilación del coseno aporta la textura de suciedad del cuarto —los picos son las zonas sucias— y sobre ella se aplica un decaimiento exponencial ligado al avance de la cobertura, que es lo que produce la curva de descontaminación descendente. Se usa coseno y no seno porque $\cos(0) = 1$: la sesión arranca en el pico de suciedad, que es el escenario realista.
Arcotan- gente	Modela la cobertura acumulada de área y, con ella, la descarga de batería , los pulsos de encoder y la deriva del RSSI . La arcotangente es monótona creciente y saturante ($\arctan(x) \rightarrow \frac{\pi}{2}$), exactamente igual que el área barrida: al principio cada metro recorrido cubre superficie nueva y al final el robot repasa lo ya limpio. Los pulsos de encoder se obtienen de su derivada, $\frac{d}{dx} \arctan(x) = \frac{1}{1+x^2}$, que decae del mismo modo que el rendimiento marginal del recorrido. Es además la serie matemáticamente más interesante, porque solo converge para $ x \leq 1$ y obliga a aplicar la identidad de reducción de argumento.

1.4 Interacción entre cliente, servidor, base de datos y dashboard

El **cliente** (hoy el generador simulado, mañana el firmware del ESP32) calcula o mide las lecturas de sus seis sensores y las envía en lotes mediante `POST /api/telemetry`, identificándose con su `deviceId`. El **servidor** —las API Routes de Next.js— valida cada carga útil con Zod, calcula las magnitudes derivadas (velocidad, ángulo girado, atasco, efectividad de limpieza), evalúa los umbrales configurados, genera los eventos que correspondan y persiste todo en la

base de datos SQLite a través de Prisma. Simultáneamente publica cada punto en un bus de eventos en memoria del que se alimenta `/api/stream`, un canal SSE (Server-Sent Events). El **dashboard** consume ese canal para actualizar KPIs, gráficos y tablas sin recargar la página, y consulta las rutas REST para las vistas históricas, los filtros y la exportación a CSV.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar una aplicación cliente-servidor de monitoreo IoT que capture, procese, almacene y visualice en tiempo real la telemetría de un robot aspirador, generando los datos de sensor mediante tres aproximaciones por series de Taylor y presentando explícitamente el error de aproximación como incertidumbre de la medición.

2.2 Objetivos específicos

1. **Aplicación cliente-servidor.** Implementar un endpoint de ingesta HTTP (POST `/api/telemetry`) con un contrato de carga útil idéntico al que usará el transporte MQTT, y un cliente que publique contra él a 2 Hz, dejando preparado el módulo suscriptor MQTT para la conexión del prototipo físico.
2. **Generación de series.** Programar tres funciones puras de aproximación de Taylor — seno, coseno y arcotangente— que devuelvan el arreglo completo de sumas parciales, y un módulo de mapeo que traduzca cada serie a una señal física con su escalado a unidades reales.
3. **Almacenamiento.** Diseñar un esquema relacional que registre, para cada lectura, el identificador de cliente, el número de registro n , el tipo de serie/señal, el valor aproximado, el valor real y los errores absoluto y relativo, junto con las entidades de sesión, evento, alerta, horario y mantenimiento.
4. **Visualización.** Construir un dashboard de cinco vistas con actualización en tiempo real por SSE, que muestre los datos de cada señal y su error, incluyendo un gráfico de convergencia del error frente a n en escala logarítmica.
5. **Control e interactividad.** Proveer comandos de operación sobre el robot, ajustes persistentes que modifiquen el comportamiento del simulador, filtros de consulta, reproducción de sesiones históricas y exportación de los datos filtrados a CSV.

3. Diseño de la solución cliente-servidor

3.1 Arquitectura propuesta

La arquitectura sigue el patrón de pasarela IoT: los dispositivos publican en un broker de mensajería, un puente suscriptor traduce los mensajes al modelo de datos del backend, y la capa de aplicación expone REST para consulta histórica y SSE para tiempo real.

ESP32 (3x HC-SR04 + GP2Y1010AU0F + 2x HC020K + ADC batería)
|

```

| MQTT sobre TLS
+-- roomba/telemetry   lote de muestras a 2 Hz
+-- roomba/event       una publicacion por evento
+-- roomba/status      retained + Last Will (online / offline)
+-- roomba/session     resumen al cerrar la sesion
+-- roomba/cmd         comandos que bajan desde el dashboard
v
BROKER MQTT   (Mosquitto local / HiveMQ Cloud)
v
PUENTE SUSCRIPTOR  src/lib/mqtt/  -- stub, mismo contrato que HTTP
v
SERVIDOR   Next.js 15 API Routes
+-- validacion Zod          src/lib/schemas.ts
+-- magnitudes derivadas    src/lib/derivados.ts
+-- motor de umbrales y eventos
+-- bus en memoria          src/lib/bus.ts
v                               v
BASE DE DATOS                SSE /api/stream
SQLite + Prisma              |
v                               v
API REST /api/* -----> DASHBOARD (React Server Components
                           + islas cliente, Recharts)

.....
RUTA ACTIVA HOY (prototipo fisico en construccion):

GENERADOR SIMULADO  src/lib/simulador/
+-- POST /api/telemetry --> exactamente el mismo endpoint y contrato
                           que usara el puente MQTT

```

[ESPACIO RESERVADO PARA CAPTURA]

Diagrama de arquitectura de bloques de la solución IoT

Figura 1: Diagrama de arquitectura de bloques de la solución IoT

Los **componentes cliente** son: (a) el firmware del ESP32, que adquiere las señales y publica; (b) el generador simulado, que lo sustituye durante el desarrollo; y (c) el navegador, que actúa como cliente de consulta y como emisor de comandos. Los **componentes servidor** son las API Routes de Next.js, el motor de reglas y umbrales, el bus de eventos y la capa de acceso a datos de Prisma.

Una decisión deliberada de diseño: el generador simulado **no escribe en la base de datos directamente**, sino que hace una petición HTTP real al endpoint de ingesta. Así se ejercita la ruta completa —serialización, validación, cálculo de derivados, persistencia y difusión SSE

— desde el primer día, y la sustitución del simulador por el hardware real no cambia una sola línea del servidor.

3.2 Flujo de datos

1. **Generación / medición.** El cliente calcula la fase x del instante actual y, para cada señal, evalúa la serie de Taylor correspondiente con n términos. En el hardware real, este paso es la lectura del eco del HC-SR04, del ADC del sensor de polvo y del conteo de los encoders.
2. **Envío.** Las muestras se agrupan en un lote y se envían como JSON con POST `/api/telemetry`. Cada lote lleva `deviceId`, `sessionId`, versión de firmware, número de secuencia y, opcionalmente, un bloque estado con las magnitudes de frecuencia media.
3. **Validación.** El servidor parsea el cuerpo con `telemetryPayloadSchema` (Zod). Un fallo devuelve 400 con cuerpo `{ error, detalle }` y el lote se descarta íntegro.
4. **Procesamiento.** Se calculan las magnitudes derivadas, se comparan con los umbrales vigentes de `Thresholds` y `Config`, y se generan los eventos que correspondan.
5. **Almacenamiento.** Cada señal de cada muestra se inserta como una fila de `Telemetry`; los eventos, en `Event`; los agregados de la sesión se actualizan en `Session`.
6. **Difusión y visualización.** Cada punto se publica en el bus y `/api/stream` lo reenvía por SSE a los navegadores conectados, que actualizan KPIs, widget de sensores, gráficos en vivo y tabla de eventos sin recargar.

3.3 Identificación del usuario correspondiente a cada cliente

Cada cliente se identifica con el campo `deviceId`, presente de forma obligatoria en toda carga útil de telemetría y en toda publicación MQTT. Ese identificador es la clave primaria del modelo `Device` y la clave foránea que enlaza sesiones, horarios y consumibles; toda fila de `Telemetry` queda asociada al dispositivo por medio de su `Session`. En el despliegue actual existe un único cliente, `robot-1`, pero el esquema no impone esa restricción: dar de alta un segundo robot es insertar una fila en `Device` sin ninguna migración.

En el transporte MQTT la identidad se refuerza por partida doble: el `deviceId` viaja en el cuerpo del mensaje y además forma parte de las credenciales del cliente ante el broker, de modo que un dispositivo no puede suplantar a otro publicando un identificador ajeno.

3.4 El número de registro n

El campo `n` cumple simultáneamente dos funciones, y esa doble lectura es central en el trabajo:

- **Como número de registro**, identifica la posición secuencial de la muestra dentro de la sesión.
- **Como número de términos**, indica cuántos sumandos de la serie de Taylor se emplearon para calcular ese valor.

Ambas están ligadas por la función `terminosEn()`, que hace crecer el número de términos con el índice de la muestra hasta un tope de 30:

```
/** Terminos de Taylor usados en la muestra i.
 * Cota superior de 30 para no desbordar el factorial. */
export function terminosEn(indice: number): number {
```

```
return Math.min(30, 6 + Math.floor(indice / 2));  
}
```

La consecuencia es directamente observable en el dashboard: a medida que avanza la sesión, el error de aproximación decae y la incertidumbre reportada de cada sensor se estrecha. El tope de 30 términos no es arbitrario: $(2k + 1)!$ desborda el rango de un double de IEEE-754 alrededor de $k = 85$, y mucho antes de eso la cancelación catastrófica entre términos alternados de magnitud creciente degrada el resultado en lugar de mejorarlo.

4. Diseño de la base de datos

4.1 Nombre y tecnología

La base de datos se llama `dev.db` y utiliza **SQLite**, accedida mediante el ORM **Prisma 6**. La elección responde a tres razones: (a) SQLite es un archivo único, sin servidor, lo que hace el proyecto reproducible con un solo comando y adecuado para una pasarela IoT doméstica que corre en el mismo equipo; (b) Prisma genera un cliente TypeScript con tipos derivados del esquema, lo que elimina el uso de `any` en toda la capa de datos; y (c) el volumen esperado —del orden de 10^4 filas de telemetría por semana— está holgadamente dentro de lo que SQLite maneja sin degradación. La migración a PostgreSQL requeriría únicamente cambiar el provider del bloque `datasource`.

4.2 Estructura de tablas

El esquema comprende once modelos. El núcleo de la evaluación son `Device`, `Session` y `Telemetry`; el resto sostiene las funciones de operación del dashboard.

Modelo	Responsabilidad
Device	El robot: identidad, modo, batería, RSSI, zona WiFi, conexión y firmware.
Config	Ajustes operativos persistentes (1:1 con Device).
Calibracion	Constantes físicas ajustables: mm por pulso, tabla RSSI→zona, línea base de PM (1:1 con Device).
Thresholds	Umbrales de alerta configurables, fila única.
Session	Una sesión de limpieza, con su resumen agregado.
Telemetry	La tabla central: una fila por señal y por muestra, con valor aproximado, valor real y errores.
Event	Eventos discretos con tipo, severidad, sensor, valor y acción tomada.
Alert	Alertas activas y resueltas derivadas de los umbrales.
Schedule / ScheduleRun	Horarios programados y su registro de cumplimiento.
Maintenance	Vida útil consumida de filtro, cepillo lateral y ciclos de batería.

4.3 Campos y tipos de datos de la tabla central

```
model Telemetry {
  id          String   @id @default(cuid())
  sessionId   String?
  session     Session? @relation(fields: [sessionId], references: [id],
                                onDelete: Cascade)

  timestamp   DateTime @default(now())
  n           Int      // numero de registro = terminos de Taylor usados
  senal       String   // tipo de serie / senal
  x           Float    // fase evaluada en la serie
  valorAproximado Float // suma parcial con n terminos
  valorReal   Float    // referencia de Math.sin / cos / atan
  errorAbsoluto Float  // |real - aproximado|
  errorRelativo Float  // errorAbsoluto / |real|
  valorFisico Float    // ya escalado a cm, mg/m3, m2 o dBm
  pulsosIzq   Int?
  pulsosDer   Int?
  voltaje     Float?
  bateriaPct  Float?
  rssi        Int?
  velocidadMs Float?
  anguloGirado Float?

  @@index([sessionId, timestamp])
  @@index([senal, n])
  @@index([timestamp])
}
```

La tabla satisface el mínimo exigido por la consigna: el identificador de usuario/cliente se alcanza por la relación `Telemetry → Session → Device.id`; el número de registro es `n`; el tipo de serie es `senal`; el dato generado es `valorAproximado` (y su forma en unidades físicas, `valorFisico`); y el error asociado se registra en sus dos formas, `errorAbsoluto` y `errorRelativo`.

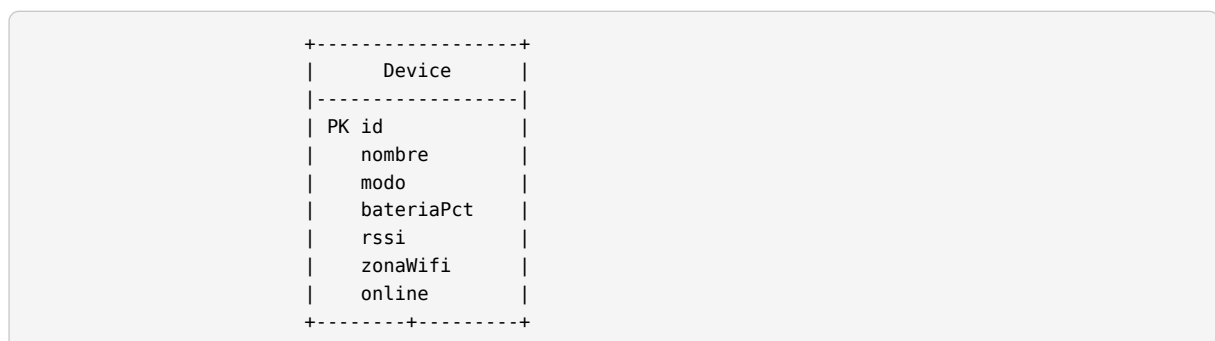
Campo	Tipo	Justificación
id	String cuid	Identificador opaco y ordenable por tiempo. Se prefiere a un entero autoincremental porque el ESP32 podrá generar identificadores sin consultar al servidor cuando publique en lote tras una reconexión.
n	Int	Conteo discreto acotado (1–200); un entero basta y permite indexar el rango de n que pide el filtro del dashboard.
senal	String	SQLite no soporta ENUM nativo. El dominio se cierra con Zod (<code>z.enum(SENALES)</code>) en el borde de la API, de modo que la garantía se mantiene sin depender del motor.
x, valorAproximado, valorReal	Float	Magnitudes continuas. Un double de 64 bits ofrece 15 dígitos significativos, muy por encima de los 10^{-12} a los que llega el error con 30 términos.
errorAbsoluto, errorRelativo	Float	Se almacenan calculados en lugar de derivarse en la consulta: el gráfico logarítmico de convergencia los ordena y

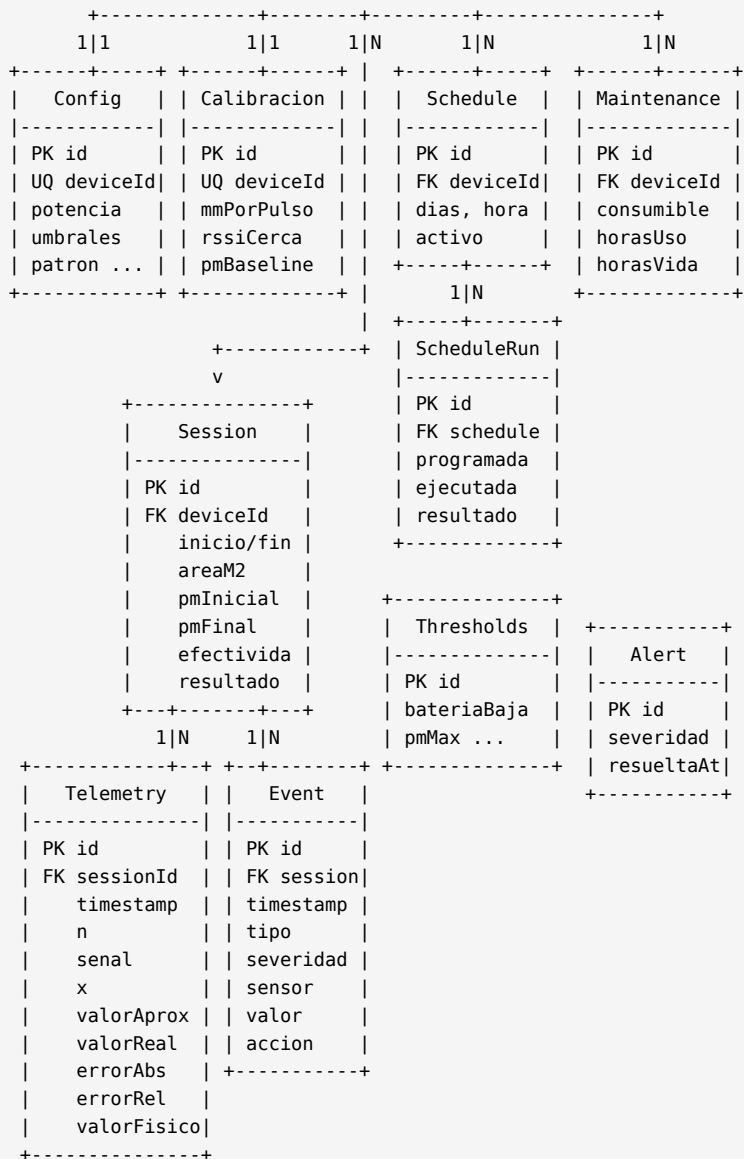
Campo	Tipo	Justificación
		agrega, y precalcularlos evita recorrer miles de filas en cada carga.
valorFisico	Float	Guardar el valor ya escalado desnormaliza deliberadamente, pero permite que la UI grafique sin repetir el mapeo en cada consulta.
timestamp	DateTime	Marca temporal de la muestra, base de la ventana deslizante en vivo y de todos los filtros por fecha.
campos opcionales	Int? / Float?	Pulsos, voltaje, batería, RSSI, velocidad y ángulo solo tienen sentido en las señales que los originan; anulables para no forzar valores ficticios.

4.4 Restricciones y reglas aplicadas

- **PRIMARY KEY** en todos los modelos (@id), con cuid() por defecto.
- **FOREIGN KEY** con integridad referencial: Telemetry.sessionId → Session.id, Session.deviceId → Device.id, ScheduleRun.scheduleId → Schedule.id.
- **ON DELETE CASCADE** en Telemetry y Event respecto de Session: borrar una sesión no deja registros huérfanos.
- **NOT NULL** implícito en todo campo sin ?: n, senal, x, los tres valores y los dos errores son obligatorios, porque una fila sin ellos no es una medición.
- **UNIQUE**: Config.deviceId y Calibracion.deviceId (relación 1:1 estricta) y la clave compuesta @@unique([deviceId, consumible]), que impide registrar dos contadores del mismo filtro para el mismo robot.
- **DEFAULT**: now() en las marcas temporales y valores de fábrica en toda la configuración y calibración, de modo que un robot recién dado de alta es operable sin configuración manual.
- **ÍNDICES**: @@index([sessionId, timestamp]) para el replay de sesión, @@index([senal, n]) para el gráfico de error frente a n, y @@index([timestamp]) para la ventana en vivo. Los tres corresponden exactamente a los tres patrones de consulta del dashboard.
- **Validación de dominio en la aplicación**: rangos físicos comprobados con Zod antes de insertar (distancias 0–500 cm, polvo 0–10 mg/m³, RSSI –120–0 dBm, n entre 1 y 200), lo que actúa como un CHECK portable entre motores.

4.5 Diagrama entidad-relación





[ESPACIO RESERVADO PARA CAPTURA]

Diagrama entidad-relación generado desde el esquema Prisma

Figura 2: Diagrama entidad-relación generado desde el esquema Prisma

5. Desarrollo de la aplicación

5.1 Tecnologías, librerías y herramientas

Herramienta	Uso en el proyecto
Next.js 15 (App Router)	Framework full-stack: Server Components para las vistas, API Routes para el servidor. Un solo despliegue para cliente y servidor.
TypeScript (modo estricto)	Tipado de extremo a extremo. El proyecto no contiene any ni @ts-ignore.
Prisma 6 + SQLite	ORM y persistencia. Migraciones versionadas y cliente tipado.
Zod 4	Validación de toda entrada: cuerpos de petición y parámetros de consulta.
Tailwind CSS v4	Sistema de estilos por utilidades, con los tokens de diseño definidos como variables CSS.
shadcn/ui	Primitivos accesibles: Card, Table, Select, Slider, Switch, Tabs, Badge, Dialog, Sheet, Toast, Tooltip y Skeleton.
Recharts 3	Todos los gráficos: líneas, áreas, histogramas, dispersión y ejes logarítmicos.
lucide-react	Iconografía de la navegación y los estados.
date-fns (locale es)	Formato y aritmética de fechas en español.
Server-Sent Events	Transporte de tiempo real del servidor al navegador.
MQTT sobre TLS	Transporte previsto entre el ESP32 y el backend; módulo suscriptor preparado.

5.2 Implementación del cliente

El cliente de ingesta vive en `src/lib/simulador/` y está completamente aislado del resto del código: la variable de entorno `SIMULADOR_ENABLED=false` lo apaga sin tocar una línea. Su ciclo es: avanzar la fase x , calcular las lecturas de las seis señales, agrupar las muestras en un lote y hacer POST al endpoint de ingesta identificándose con su `deviceId`.

```
// Lecturas de un instante. `n` es el numero de terminos de Taylor usados:
// crece con el numero de registro, de modo que la incertidumbre decae
// visiblemente a lo largo de la sesion.
export function lecturas(x: number, n: number): Record<Senal, LecturaSenal> {
  const frontal = aproximar('sin', x + DESFASE.frontal, n);
  const izq     = aproximar('sin', x + DESFASE.izquierda, n);
  const der     = aproximar('sin', x + DESFASE.derecha, n);
  const pm      = aproximar('cos', x, n);
  const cob     = aproximar('atan', x, n);
  // Fraccion del area objetivo ya barrida; alimenta el decaimiento del polvo.
  const progreso = cobertura(cob) / 18;
  ...
}
```

El **identificador del usuario** viaja en cada lote y es el que el servidor usa para resolver el dispositivo. Contrato JSON de la carga útil de telemetría:

```
{
  'deviceId': 'robot-1',
  'sessionId': 'clx8f2k...',
  'fw': '0.1.0',
  'seq': 1287,
  'muestras': [
    {
      't': '2026-09-10T20:31:04.500Z',
      'n': 18,
      'x': 3.42,
      'sensores': {
        'distanciaFrontalCm': 47.3,
        'distanciaIzquierdaCm': 132.8,
        'distanciaDerechaCm': 88.1,
        'densidadPolvoMgM3': 0.214,
        'pulsosIzq': 11,
        'pulsosDer': 13
      },
      'aprox': {
        'DIST_FRONTAL': { 'valorAproximado': -0.6601, 'valorReal': -0.6600 }
      }
    }
  ],
  'estado': {
    'modo': 'LIMPIANDO',
    'voltajeBateria': 15.1,
    'nivelBateriaPct': 64.5,
    'rssiDbm': -58,
    'zonaWifi': 'MEDIA',
    'distanciaAcumuladaM': 214.7,
    'areaEstimadaM2': 38.6,
    'potenciaSuccion': 'NORMAL'
  }
}
```

Nota: las comillas se muestran simples por legibilidad tipográfica; en el payload real son comillas dobles JSON.

El bloque `aprox` es el único campo que el ESP32 real no enviará: el hardware mide, no aproxima. Por eso es opcional en el esquema, y su ausencia solo deja vacío el gráfico de convergencia sin afectar a ninguna otra vista.

5.3 Implementación del servidor

El servidor expone diecinueve rutas bajo `/app/api`. Toda entrada pasa por Zod y todo error se devuelve con el código HTTP correcto y cuerpo `{ error, detalle }`.

Ruta	Método	Función
<code>/api/telemetry</code>	POST	Ingesta: valida, calcula derivados y errores, evalúa umbrales, dispara eventos y persiste.
<code>/api/telemetry</code>	GET	Serie temporal filtrable por señal, sesión, rango de n y de fechas. Paginada y ordenable.
<code>/api/device</code>	GET	Estado del robot: modo, batería, RSSI, zona, uptime, última conexión.
<code>/api/device/cmd</code>	POST	<code>start / pause / stop / dock / reboot / locate</code> .

Ruta	Método	Función
/api/device/config	GET, PUT	Ajustes persistentes del robot.
/api/device/calibration	GET, PUT	Pulsos por metro, tabla RSSI→zona, línea base de PM.
/api/schedules	GET, POST	Listar y crear horarios.
/api/schedules/[id]	PUT, DELETE	Editar, activar/desactivar y borrar.
/api/sessions	GET	Historial con resumen por sesión.
/api/sessions/[id]	GET	Detalle con toda la telemetría para el replay.
/api/events	GET	Eventos filtrables por tipo, severidad y fecha.
/api/alerts	GET	Alertas activas y resueltas.
/api/alerts/thresholds	GET, PUT	Umbrales configurables.
/api/maintenance	GET, POST	Vida útil de consumibles y reset.
/api/stats	GET	KPIs agregados del dashboard.
/api/stream	GET	Canal SSE con telemetría y eventos en vivo.
/api/simulator	POST	start / stop / velocidad del generador.
/api/export	GET	CSV de los datos filtrados.

5.4 Implementación de las tres series matemáticas

Las tres funciones viven en `src/lib/series.ts`, son puras y devuelven el **arreglo completo de sumas parciales** $S_0, S_1, \dots, S_{n-1}$, no solo el resultado final; esto es lo que permite graficar la convergencia término a término.

```
/**
 * sin(x) = S (-1)^k * x^(2k+1) / (2k+1)!
 * Converge para todo x, pero con |x| grande hay cancelacion catastrófica:
 * el mapeo de sensores mantiene x en un rango moderado por esa razon.
 */
export function sinTaylor(x: number, terminos: number): number[] {
  const sumas: number[] = [];
  let suma = 0;
  for (let k = 0; k < terminos; k++) {
    suma += ((-1) ** k * x ** (2 * k + 1)) / factorial(2 * k + 1);
    sumas.push(suma);
  }
  return sumas;
}

/** cos(x) = S (-1)^k * x^(2k) / (2k)! */
export function cosTaylor(x: number, terminos: number): number[] {
```

```

const sumas: number[] = [];
let suma = 0;
for (let k = 0; k < terminos; k++) {
  suma += ((-1) ** k * x ** (2 * k)) / factorial(2 * k);
  sumas.push(suma);
}
return sumas;
}

/**
 * atan(x) = S (-1)^k * x^(2k+1) / (2k+1) (serie de Gregory-Leibniz)
 * OJO: solo converge para |x| <= 1. Para |x| > 1 se usa la identidad
 * atan(x) = sign(x)*PI/2 - atan(1/x), aplicando la serie al reciproco.
 */
export function atanTaylor(x: number, terminos: number): number[] {
  const fuera = Math.abs(x) > 1;
  const u = fuera ? 1 / x : x;
  const sumas: number[] = [];
  let suma = 0;
  for (let k = 0; k < terminos; k++) {
    suma += ((-1) ** k * u ** (2 * k + 1)) / (2 * k + 1);
    sumas.push(fuera ? Math.sign(x) * (Math.PI / 2) - suma : suma);
  }
  return sumas;
}

```

Dos detalles numéricos merecen comentario. Primero, la **reducción de argumento de la arcotangente**: la serie de Gregory–Leibniz diverge para $|x| > 1$, y como la fase de la simulación llega hasta $x = 6$, sin la identidad $\arctan(x) = \text{sign}(x) \cdot \frac{\pi}{2} - \arctan(\frac{1}{x})$ la señal de cobertura sería inutilizable. Segundo, la **cancelación catastrófica** de seno y coseno: aunque convergen para todo x , con $|x|$ grande los términos intermedios superan ampliamente al resultado y la resta entre ellos destruye dígitos significativos; el mapeo de sensores mantiene x en un rango moderado precisamente por eso.

La envoltura `aproximar()` calcula en una sola pasada la aproximación, el valor real de referencia y ambos errores:

```

export function aproximar(fn: Fn, x: number, terminos: number): Aproximacion {
  const { serie, real } = IMPL[fn];
  const sumasParciales = serie(x, Math.max(1, terminos));
  const valorAproximado = sumasParciales[sumasParciales.length - 1];
  const valorReal = real(x);
  const errorAbsoluto = Math.abs(valorReal - valorAproximado);
  // Si el valor real es ~0, el error relativo se normaliza contra 1
  // para evitar una division que dispararia el cociente al infinito.
  const denominador = Math.abs(valorReal) < 1e-9 ? 1 : Math.abs(valorReal);
  return {
    valorAproximado, valorReal, errorAbsoluto,
    errorRelativo: errorAbsoluto / denominador,
    sumasParciales,
  };
}

```

Mapeo de las series a señales físicas

Señal	Serie	Escalado	Rango
Distancia frontal	$\sin(x)$	$110 + 95 \sin(x)$	15 – 205 cm

Señal	Serie	Escalado	Rango
Distancia izquierda	$\sin(x + 1.1)$	ídem	15 – 205 cm
Distancia derecha	$\sin(x + 2.3)$	ídem	15 – 205 cm
Densidad de polvo	$\cos(x)$	$0.25 + 0.22 \cos(x)$	$0.03 - 0.47 \text{ mg/m}^3$
Cobertura y batería	$\arctan(x)$	$18 \arctan(x) / (\pi/2) - 18 \text{ m}^2$	
Pulsos de encoder	derivada de arctan	$1/(1 + x^2)$	crecen con la cobertura
RSSI	fase lenta de arctan	$-35 - 43a$	-35 a -78 dBm

El recorte al rango útil del HC-SR04 no es cosmético: con pocos términos la serie puede dispararse fuera de todo rango físico, y el sensor real tampoco reportaría un valor imposible.

```
/**
 * Distancia ultrasonica: 110 + 95*sin(x + desfase) -> 15-205 cm.
 * Se recorta al rango util del HC-SR04 igual que haria el sensor real:
 * con pocos terminos la serie puede dispararse fuera de todo rango fisico.
 */
function distancia(aprox: Aproximacion): number {
  return Math.max(15, Math.min(205, 110 + 95 * aprox.valorAproximado));
}
```

5.5 Magnitudes derivadas en el backend

Ninguna de estas magnitudes se mide: todas se calculan a partir de los encoders, del ADC de batería o del sensor de polvo. El robot no tiene IMU, ni sensor de corriente, ni de temperatura, y ninguna métrica del sistema depende de ellos.

Magnitud	Definición
Velocidad lineal	$v = \frac{((p_{izq} + p_{der})/2) \cdot 10.2 \text{ mm}}{\Delta t}$
Ángulo girado	$\theta = \frac{d_{der} - d_{izq}}{150 \text{ mm}} - \text{odometría diferencial, sin IMU; acumula deriva.}$
Distancia recorrida	Promedio de ambas ruedas por los mm de cada pulso.
Área barrida	Distancia lineal por 0.18 m de ancho efectivo del cepillo.
Atascado	Motores comandados ON y pulsos ≈ 0 durante más de 3 s.
Rueda trabada	Una rueda cuenta pulsos y la otra permanece en cero.
Caída de tensión anómala	El voltaje cae más rápido de lo esperado. Sustituye al sensor de corriente descartado: delata un cepillo trabado con pelo, que exige más corriente al motor.

Magnitud	Definición
Efectividad de limpieza	$(PM_{\text{inicial}} - PM_{\text{final}}) / PM_{\text{inicial}} \times 100$
Zona sucia	Pico de PM por encima de $1.5 \times$ la media móvil (factor configurable entre 1.2 y 3).

```

/**
 * Angulo girado por odometria diferencial, en radianes:
 * t = (distancia_derecha - distancia_izquierda) / distancia_entre_ejes
 * Sin IMU esta es la unica fuente de orientacion, y acumula deriva.
 */
export function anguloGirado(
  pulsosIzq: number, pulsosDer: number,
  mmPorPulso = HARDWARE.mmPorPulso,
  ejesMm = HARDWARE.distanciaEjesMm,
): number {
  return ((pulsosDer - pulsosIzq) * mmPorPulso) / ejesMm;
}

/**
 * Caída de tensión anómala. Sustituye al sensor de corriente que se descarto:
 * si el voltaje cae mas rapido de lo esperado, el motor esta exigiendo mas
 * corriente de la normal, tipicamente por un cepillo trabado con pelo.
 */
export function caidaTensionAnomala(
  voltajeAnterior: number, voltajeActual: number,
  deltaS: number, umbralVPorMin: number,
): boolean {
  if (deltaS <= 0) return false;
  const caidaPorMinuto = ((voltajeAnterior - voltajeActual) / deltaS) * 60;
  return caidaPorMinuto > umbralVPorMin;
}

```

Las constantes de calibración se derivan de la geometría real del prototipo y son editables desde la interfaz, porque el hardware nunca coincide con el papel: una rueda impresa en 3D no mide exactamente 65 mm y el disco ranurado puede perder un pulso por vuelta.

```

export const HARDWARE = {
  /** Disco ranurado HC020K de 20 ranuras sobre rueda de 65 mm de diametro:
   * perimetro = PI * 65 mm ~ 204.2 mm -> 204.2 / 20 ~ 10.2 mm por pulso. */
  mmPorPulso: 10.2,
  /** Separacion entre ruedas, usada en la odometria diferencial. */
  distanciaEjesMm: 150,
  /** Ancho barrido por el cepillo, usado para estimar area cubierta. */
  anchoCepilloM: 0.18,
  /** Pulsos necesarios para recorrer 1 m: 1000 / 10.2 ~ 98.04 */
  pulsosPorMetro: 1000 / 10.2,
} as const;

```

5.6 Inserción de datos: conexión, operación y manejo de errores

La conexión a la base de datos se establece una sola vez por proceso mediante un cliente Prisma memorizado en el ámbito global, patrón necesario en desarrollo porque la recarga en caliente de Next.js reevalúa los módulos y, sin él, agotaría el pool de conexiones.

El manejo de errores opera en tres niveles: (a) **validación de entrada** con Zod, que rechaza el lote completo con 400 y detalle del campo ofensor; (b) **transacción por lote**, de modo que

un fallo a mitad de la escritura no deja una sesión con telemetría parcial; y (c) **tolerancia del cliente**, que reintenta el envío con retroceso exponencial ante un 5xx, del mismo modo que lo hará el ESP32 tras una reconexión WiFi.

```
export async function POST(req: Request) {
  const parsed = telemetryPayloadSchema.safeParse(await req.json());
  if (!parsed.success) {
    return Response.json(
      { error: 'PAYLOAD_INVALIDO', detalle: parsed.error.issues },
      { status: 400 },
    );
  }
  // ... calculo de derivados, evaluacion de umbrales y generacion de eventos
  await prisma.$transaction([
    prisma.telemetry.createMany({ data: filas }),
    prisma.event.createMany({ data: eventos }),
    prisma.session.update({ where: { id: sessionId }, data: resumen }),
  ]);
  // Difusion a los clientes SSE conectados.
  for (const punto of puntos) bus.publicar({ tipo: 'telemetria', datos: punto });
  return Response.json({ ok: true, insertadas: filas.length }, { status: 201 });
}
```

5.7 Lectura y recuperación de datos

Las consultas se construyen a partir de parámetros validados también con Zod (telemetryQuerySchema), que aplica coerción de tipos sobre la cadena de consulta y rechaza filtros mal formados antes de tocar la base de datos. El resultado se pagina y se ordena por la columna solicitada, y las mismas condiciones alimentan la exportación a CSV, de modo que el archivo descargado refleja exactamente lo que el usuario está viendo en pantalla.

Las vistas históricas se resuelven en **React Server Components**: la consulta ocurre en el servidor y al navegador solo llega el HTML ya resuelto, sin exponer la capa de datos ni enviar el ORM al cliente. Solo los componentes que requieren interactividad —gráficos, controles, suscripción SSE— se marcan con "use client".

5.8 Otras funciones implementadas

- **Motor de reglas y umbrales** que compara cada muestra con la configuración vigente y genera los once tipos de evento, clasificados en severidades INFO, ADVERTENCIA y CRITICO.
- **Comandos de operación** (start, pause, stop, dock, reboot, locate) con confirmación por toast cuando el robot acusa recibo.
- **Asistentes de calibración**: tres puntos para la tabla RSSI→zona y avance de un metro para el conteo de pulsos por metro.
- **Planificador de horarios** con registro de cumplimiento: programada, ejecutada, retraso, resultado y motivo de omisión.
- **Contadores de consumibles** con vida útil y reset individual por filtro, cepillo lateral y ciclos de batería.
- **Seed reproducible** con un PRNG determinista (mulberry32), que puebla la base con 12 sesiones históricas repartidas en tres semanas, 2500 registros de telemetría, 180 eventos, 3 horarios y una sesión en curso, de modo que el dashboard se ve poblado desde el primer arranque.

- **Pruebas unitarias** de las series y de las magnitudes derivadas (`series.test.ts`, `derivados.test.ts`), que verifican la convergencia contra `Math.sin`, `Math.cos` y `Math.atan` y la detección de atasco y rueda trabada.

6. Desarrollo del dashboard

6.1 Diseño y herramientas

El dashboard se construyó con los mismos Server Components de Next.js, estilos con Tailwind CSS v4, primitivos accesibles de shadcn/ui y gráficos con Recharts. No se usó Flask ni Plotly: al estar cliente y servidor en el mismo framework, la vista consulta la base de datos sin una capa HTTP intermedia y el tiempo real se resuelve con SSE nativo.

La estética es la de un panel de instrumentación industrial: densidad de información alta pero respirada, sin decoración. Tres principios rigen el diseño. **El dato es el héroe**: bordes de 1 px en lugar de sombras, superficies planas, cero gradientes. **El color solo comunica estado**: la interfaz es neutral en escala de grises y el color aparece únicamente para distinguir señales o señalar un estado. **Una sola cosa llama la atención por pantalla**: en el panel principal ese acento es el widget de sensores ultrasónicos.

Aspecto	Decisión
Temas	Oscuro por defecto y claro, ambos con paleta completa de tokens CSS; la preferencia persiste entre sesiones y funciona en todas las páginas.
Tipografía	Inter para la interfaz, JetBrains Mono para valores en vivo, tablas y ejes. Números con <code>tabular-nums</code> , para que no bailen al actualizarse.
Acentos semánticos	Cian para el sensor frontal, índigo para el izquierdo, violeta para el derecho, ámbar para el polvo, verde para la batería; verde, ámbar y rojo para ok, advertencia y crítico.
Layout	Sidebar colapsable de 240 px con estado persistente; rejilla de 12 columnas con <code>gap: 16px</code> , que colapsa a 6 en tablet y a 1 en móvil.
Estados	Carga con skeletons con la forma del contenido final, nunca spinners; vacío con acción que lo resuelve; error con detalle plegable y reintento; banner discreto de desconexión.
Accesibilidad	Nivel AA: roles ARIA, foco visible, contraste suficiente y navegación por teclado en tablas y formularios.

6.2 Páginas y visualizaciones

/ — Panel principal

Barra superior fija con el nombre del robot, chip de estado con color, badge «EN VIVO» con punto pulsante, hora de la última telemetría y los cuatro botones de comando. Debajo, una fila de cuatro tarjetas de KPI con sparkline de fondo: batería con minutos restantes estimados, efectividad de limpieza, obstáculos evitados hoy y área estimada con su delta respecto a ayer.

Los botones de comando se habilitan según el estado del robot, y los deshabilitados se muestran atenuados, nunca ocultos:

Botón	Habilitado cuando el robot está
Iniciar limpieza	EN BASE o PAUSADO
Pausar	LIMPIANDO
Detener	LIMPIANDO o PAUSADO
Volver a base	LIMPIANDO o PAUSADO

Componentes principales de la vista:

- **Widget de sensores ultrasónicos** — el elemento visual más distintivo: tres barras radiales dispuestas en abanico (izquierda, frontal, derecha) con la distancia actual en centímetros. La barra se tiñe de ámbar bajo el umbral de frenado configurado y de rojo bajo 10 cm. Debajo, el conteo de detecciones de cada sensor en la sesión.
- **Gráfico de telemetría en vivo** — ventana deslizante de los últimos 120 s, con las tres distancias en el eje izquierdo y la densidad de polvo en el derecho. Marcas verticales tenues donde ocurrieron maniobras de evasión, leyenda clicable para aislar señales y tooltip con valores exactos.
- **Curva de descontaminación** — el PM de la sesión desde su pico inicial hacia abajo, con línea punteada en el umbral objetivo y área sombreada bajo la curva. Es el gráfico que demuestra que el robot efectivamente limpia, y el único donde se permite relleno sólido.
- **Tarjeta de odometría** — distancia recorrida, velocidad actual, pulsos acumulados por rueda y una barra de simetría izquierda/derecha que permite detectar una rueda trabada de un vistazo.
- **Gauge de batería** — anillo de progreso con la curva de descarga de las últimas dos horas y línea punteada en el umbral de retorno.
- **Indicador de zona WiFi** — tres segmentos horizontales (Cerca, Media, Lejos del router) con el actual resaltado, el RSSI en dBm y la etiqueta explícita «estimación por intensidad de señal».
- **Próxima limpieza** — cuenta regresiva, horario programado y botón para saltarla esta vez.
- **Tabla de eventos recientes** — los últimos 20, con hora, tipo con badge de color por severidad, sensor, valor y acción tomada.

[ESPACIO RESERVADO PARA CAPTURA]

Panel principal en tema oscuro con el robot en sesión activa

Figura 3: Panel principal en tema oscuro con el robot en sesión activa

[ESPACIO RESERVADO PARA CAPTURA]

Widget de sensores ultrasónicos con el sensor frontal en estado crítico

Figura 4: Widget de sensores ultrasónicos con el sensor frontal en estado crítico

[ESPACIO RESERVADO PARA CAPTURA]

Curva de descontaminación y tarjeta de odometría

Figura 5: Curva de descontaminación y tarjeta de odometría

/telemetry

Vista analítica con filtros por rango de fechas, sesión, señal y rango de n ; gráfico temporal multi-señal con zoom por brush; histograma de distancias por sensor con las tres series superpuestas —que revela si el robot vive pegado a las paredes o en espacio abierto—; y la tabla completa paginada con timestamp, sesión, n , señal, valor medido, valor real, error absoluto y error relativo, ordenable por cualquier columna y exportable a CSV.

El gráfico central de esta página es el de **precisión de la estimación según el número de muestras (n)**: error frente a n con eje Y logarítmico y una línea por señal. La escala logarítmica es indispensable porque el error cae varios órdenes de magnitud —de 10^{-1} a 10^{-12} — y en escala lineal todo salvo los primeros puntos quedaría aplastado contra el eje. Se complementa con un diagrama de dispersión de error frente a n coloreado por señal.



Figura 6: Gráfico de precisión de la estimación según el número de muestras (n), eje logarítmico

[ESPACIO RESERVADO PARA CAPTURA]

Tabla de telemetría con valor aproximado, valor real y errores

Figura 7: Tabla de telemetría con valor aproximado, valor real y errores

/sesiones

Lista con fecha, duración, área estimada, PM inicial y final, efectividad, batería consumida, obstáculos y resultado (completada, batería baja, cancelada o atascada), acompañada de barras comparativas de efectividad y de m^2/min entre las últimas diez sesiones. El detalle /sesiones/

[id] ofrece el **replay** de la telemetría con reproductor (play, pausa y velocidades $\times 1$, $\times 2$ y $\times 4$), una línea de tiempo horizontal de eventos, tarjetas de resumen y la tabla de datos crudos.



Figura 8: Historial de sesiones con barras comparativas de efectividad



Figura 9: Detalle de sesión con reproductor de telemetría y timeline de eventos

/horarios

Calendario semanal con los bloques de limpieza programados y un panel lateral (Sheet) para crear y editar, con chips de días de la semana, hora de inicio, duración máxima, potencia, patrón y un toggle para saltar la ejecución si la batería está por debajo de un umbral. Una tabla de cumplimiento registra programada, ejecutada, retraso, resultado y motivo de omisión, y un anillo de progreso resume el porcentaje de cumplimiento del mes.



Figura 10: Calendario semanal de horarios y panel de edición

/ajustes

Organizada en pestañas: **Robot** (todos los sliders, selects y toggles, con guardado que publica en cmd/config y confirma al acusar recibo), **Alertas** (umbrales editables y lista de alertas activas y resueltas), **Mantenimiento** (barras de vida útil con reset por consumible), **Calibración** (pulsos por metro, tabla RSSI→zona, línea base de PM y los dos asistentes) y **Conectividad** (RSSI histórico, uptime, tasa de pérdida de paquetes y últimas reconexiones).

Ajustes configurables de la pestaña Robot:

Ajuste	Control	Rango
Potencia de succión	Slider de 3 pasos	Eco / Normal / Turbo
Velocidad	Slider	0.1 – 0.4 m/s
Umbral de frenado frontal	Slider	10 – 40 cm
Umbral de frenado lateral	Slider	5 – 25 cm
Duración de giro en evasión	Slider	0.2 – 1.5 s
Patrón de limpieza	Select	Zigzag / Espiral / Perímetro / Aleatorio
Duración máxima de sesión	Input numérico	10 – 120 min
Batería mínima para retornar	Slider	10 – 40 %
Umbral de zona sucia	Slider	1.2× – 3× la media
Segunda pasada en zona sucia	Toggle	On / Off
Terminar al alcanzar limpieza	Toggle + umbral	PM objetivo en mg/m ³
Frecuencia de telemetría	Select	0.5 / 1 / 2 Hz
Modo silencioso	Toggle + rango horario	p. ej. 22:00 – 07:00

Ajuste	Control	Rango
Nombre del robot	Input de texto	libre



Figura 11: Pestaña de ajustes del robot con los controles de configuración

6.3 Visualización del error por señal

El error de aproximación se presenta al usuario como **incertidumbre de la medición del sensor**, en dos formas simultáneas: en unidades físicas junto al valor (por ejemplo, 47.3 ± 0.4 cm) y en su forma numérica cruda en la tabla de telemetría, con los errores absoluto y relativo. Esta traducción es lo que da sentido de dominio al ejercicio matemático: un error de 3×10^{-3} en la suma parcial del seno no significa nada para el operador del robot, pero ± 0.3 cm sí.

6.4 Actualización en tiempo real

La actualización se resuelve con **Server-Sent Events** sobre `/api/stream`, y no con WebSockets, porque el flujo es unidireccional: el servidor empuja y el cliente solo escucha. SSE se reconecta solo, atraviesa proxies HTTP sin configuración adicional y no requiere librería alguna en el navegador.

Aspecto	Implementación
Frecuencia	2 Hz en telemetría (configurable a 0.5, 1 o 2 Hz) y publicación inmediata por evento.
Manejo de datos	Buffer en cliente de 240 puntos —120 s a 2 Hz— con descarte de los más antiguos, para que la memoria no crezca sin límite en una sesión larga.
Conexión	Reconexión automática con retroceso exponencial y respaldo por sondeo cada 5 s si SSE falla tres veces consecutivas.
Transición visual	Los números de los KPI hacen una transición breve al cambiar, sin conteo animado, que distraería de la lectura.

El acoplamiento entre la ingesta y el canal en vivo es un bus de eventos en memoria, deliberadamente simple:

```
/**
 * Bus de eventos en proceso. La ruta de ingesta publica aqui y /api/stream
 * reenvia a los clientes por SSE. Es deliberadamente in-memory: con un solo
 * robot y un solo proceso de Next no hace falta Redis.
 */
```

6.5 Interactividad

- **Filtros** por rango de fechas, sesión, señal y rango de n , encadenables entre sí.
- **Leyenda clicable** en los gráficos multi-señal para aislar o superponer curvas.
- **Zoom por brush** sobre el eje temporal.
- **Ordenamiento** por cualquier columna en todas las tablas, con paginación.
- **Reproductor de sesión** con play, pausa y velocidades $\times 1$, $\times 2$ y $\times 4$.
- **Controles de configuración** —sliders, selects y toggles— cuyos cambios persisten y modifican efectivamente el comportamiento del simulador.
- **Exportación a CSV** que respeta los filtros aplicados en el momento de la descarga.
- **Alternancia de tema** claro/oscuro con persistencia.
- **Acciones secundarias**: calibrar zonas WiFi (asistente de 3 puntos), calibrar encoders (avanzar 1 m y contar pulsos), marcar mantenimiento hecho y reiniciar el ESP32.

7. Pruebas y resultados

7.1 Comunicación cliente-servidor

Se verificó el ciclo completo enviando lotes al endpoint de ingesta y comprobando la respuesta 201 con el número de filas insertadas, así como el rechazo con 400 y detalle del campo ofensor ante cargas útiles mal formadas (distancia fuera del rango 0–500 cm, n fuera de 1–200, deviceId ausente).

[ESPACIO RESERVADO PARA CAPTURA]
Petición POST a /api/telemetry y respuesta del servidor

Figura 12: Petición POST a /api/telemetry y respuesta del servidor



Figura 13: Rechazo con código 400 y detalle Zod ante una carga útil inválida

7.2 Generación y almacenamiento de las tres series

Las pruebas unitarias de `series.test.ts` comparan las sumas parciales contra `Math.sin`, `Math.cos` y `Math.atan`, verificando que el error decrece con el número de términos y que la reducción de argumento de la arcotangente mantiene la convergencia para $|x| > 1$.



Figura 14: Ejecución de las pruebas unitarias de series y magnitudes derivadas

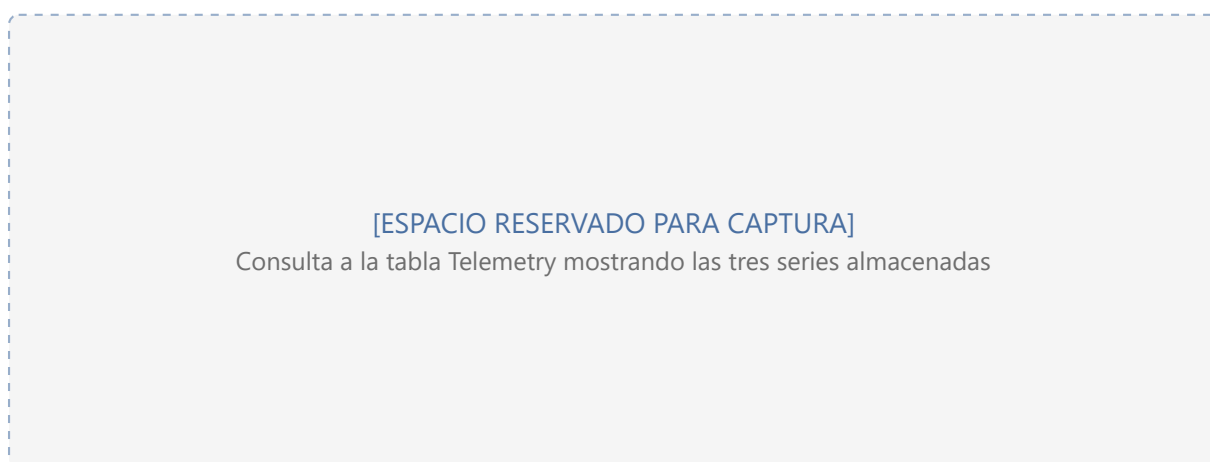


Figura 15: Consulta a la tabla Telemetry mostrando las tres series almacenadas

Convergencia observada. El comportamiento es el esperado para series alternadas: el error queda acotado por el primer término omitido. Seno y coseno convergen factorialmente y alcanzan la precisión de la máquina con pocas decenas de términos; la arcotangente, cuyo denominador crece solo linealmente, converge de forma notablemente más lenta —es la misma razón por la que la serie de Leibniz para π es célebremente ineficiente— y es precisamente ese

contraste lo que hace legible el gráfico de convergencia: tres pendientes claramente distintas en el mismo eje logarítmico.

7.3 Registros por usuario y por valor de n

Se comprobó que toda fila de telemetría es trazable hasta su cliente por la cadena Telemetry → Session → Device.id, y que el filtro por rango de n devuelve exactamente las muestras cuya aproximación usó ese número de términos.



Figura 16: Registros filtrados por deviceId y por rango de n

7.4 Capturas del dashboard con los datos y el error



Figura 17: Panel principal poblado con datos en vivo

[ESPACIO RESERVADO PARA CAPTURA]

Error de aproximación presentado como incertidumbre de la medición

Figura 18: Error de aproximación presentado como incertidumbre de la medición

7.5 Análisis de los resultados

Los resultados confirman las tres hipótesis de diseño. Primero, que **un error matemático de aproximación es un modelo honesto de la incertidumbre de un sensor**: ambos describen la distancia entre un valor reportado y un valor verdadero inaccesible, y ambos se reducen invirtiendo más recursos —términos de la serie en un caso, muestras promediadas en el otro. Segundo, que **la forma de la serie condiciona la calidad de la señal simulada**. Las señales derivadas de seno y coseno alcanzan precisión de máquina temprano y su incertidumbre deja de ser visible pasadas las primeras muestras; la cobertura, derivada de la arcotangente, conserva una incertidumbre apreciable durante toda la sesión. Esto reproduce algo cierto del hardware real: un ultrasónico es un sensor preciso, mientras que una estimación de área acumulada por odometría arrastra deriva y nunca deja de ser una estimación.

Tercero, que **enrutar el simulador por el mismo endpoint HTTP que usará el ESP32 fue la decisión más rentable del proyecto**. Todo el camino —serialización, validación, cálculo de derivados, transacción, difusión SSE— quedó ejercitado desde el primer día, y la sustitución del generador por el hardware real no exige cambios en el servidor ni en la interfaz.

Como limitación, debe señalarse que la estimación de zona WiFi por RSSI es deliberadamente gruesa. El RSSI en interiores acumula un error de varios metros por paredes, muebles y reflexiones multitrayecto, de modo que no se usa para calcular distancia: solo clasifica en tres zonas calibradas por el usuario, y la interfaz lo etiqueta siempre como estimación por intensidad de señal, nunca como posición.

8. Conclusiones

1. **Se cumplió el objetivo general.** La aplicación cliente-servidor captura, valida, procesa, almacena y visualiza telemetría en tiempo real, con un ciclo completo verificado desde la generación de la muestra hasta su aparición en el gráfico en vivo sin recargar la página.
2. **Las tres series de Taylor cumplieron una doble función.** Además de satisfacer el requisito matemático, proporcionaron un modelo de señal físicamente coherente: la periodicidad del seno para las distancias, la oscilación decreciente del coseno para la suciedad y

la saturación de la arcotangente para el área cubierta. La elección de la serie según la forma del fenómeno resultó más valiosa que una asignación arbitraria.

3. **El error de aproximación es comunicable como incertidumbre de sensor.** Presentarlo en unidades físicas junto al valor medido convirtió un dato abstracto en información útil para el operador, y el gráfico logarítmico de error frente a n hace visible la convergencia de las tres series con sus tres pendientes distintas.
4. **El diseño de la base de datos resistió las tres consultas del dashboard.** Los índices sobre (sessionId, timestamp), (senal, n) y (timestamp) corresponden exactamente a los patrones de acceso del replay, del gráfico de convergencia y de la ventana en vivo, y ninguna vista requirió recorrer la tabla completa.
5. **La separación entre simulación y aplicación quedó verificada.** Toda la simulación vive aislada en src/lib/simulador/ y se apaga con una variable de entorno; con SIMULADOR_ENABLED=false la aplicación sigue funcional, mostrando los datos históricos y esperando datos reales, lo que demuestra que el frontend no depende del origen de los datos.
6. **El trabajo dejó preparada la integración del hardware.** El contrato de carga útil es idéntico para HTTP y MQTT, los tópicos están definidos con estado retenido y Last Will, y el módulo suscriptor solo requiere las credenciales del broker. Las constantes de calibración son editables desde la interfaz, porque el hardware nunca coincide con el papel.
7. **Limitaciones reconocidas.** No hay autenticación ni multi-tenant, el bus de eventos es en memoria —lo que ata el sistema a un único proceso de servidor— y la estimación de zona WiFi es cualitativa por naturaleza. Las tres son consecuencias asumidas del alcance definido, y ninguna requiere rediseñar el esquema de datos para resolverse.

9. Anexos y entregables

- **Repositorio GitHub:** _____
Acceso configurado para la docente: pamela.valenzuela.f@ucb.edu.bo.
- **Código fuente completo**, incluyendo el esquema Prisma, las migraciones, el script de seed y las pruebas unitarias.
- **Archivos generados por la aplicación:** base de datos dev.db poblada y exportaciones CSV de ejemplo.
- **Capturas de pantalla** de las cinco vistas del dashboard, en tema claro y oscuro.
- **Diagramas:** arquitectura de bloques IoT y entidad-relación de la base de datos.
- **Instrucciones de ejecución:**

```
npm install
npx prisma migrate dev      # crea dev.db y aplica el esquema
npx prisma db seed          # puebla 12 sesiones, ~2500 muestras, ~180 eventos
npm run dev                 # http://localhost:3000
```

- El informe y los archivos generados deben ser comprimidos en un archivo .rar o .zip y subidos al LMS por cada estudiante.

- **Exposición y defensa:** lunes 07 y viernes 11 de septiembre de 2026, en horario de clases.

Lista de verificación antes de la entrega

Criterio	Verificación
Aplicación cliente-servidor documentada	☐
Tres tipos de series matemáticas implementados y explicados	☐
Identificador de usuario/cliente registrado	☐
Número de registro (n) considerado	☐
Base de datos y diagrama incluidos	☐
Dashboard con datos por usuario y error	☐
Gráficos relevantes incluidos	☐
Repositorio GitHub accesible para la docente	☐
Archivo .rar o .zip preparado para LMS	☐

Verificación técnica del sistema

Criterio de aceptación	Estado
<code>npm run dev</code> levanta la app sin errores y el dashboard muestra datos desde el primer segundo	☐
Los cuatro botones de comando responden y se habilitan según el estado	☐
El widget de sensores ultrasónicos cambia de color al cruzar los umbrales configurados	☐
El gráfico en vivo se actualiza por SSE sin recargar la página	☐
El gráfico de error vs n usa eje logarítmico y muestra la convergencia de las tres series	☐
Todos los ajustes persisten y se reflejan en el comportamiento del simulador	☐
Los horarios se crean, editan, activan y borran correctamente	☐
El replay de sesión reproduce la telemetría a distintas velocidades	☐
La exportación a CSV respeta los filtros aplicados	☐
El tema claro/oscuro funciona en todas las páginas y persiste	☐

Criterio de aceptación	Estado
Ninguna página contiene mapa, planta del cuarto ni trayectoria 2D	☐
Ninguna métrica depende de IMU, sensor de corriente ni sensor de temperatura	☐
Apagar el simulador deja la app funcional, esperando datos reales	☐

Capturas pendientes de insertar. Cada marco punteado reserva el espacio de una captura. Para insertarla basta pasar la ruta del archivo al parámetro archivo:

```
#captura("Panel principal en tema oscuro", archivo: "capturas/panel.png")
```